

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPILER DESIGN

COURSE CODE: CSA1412

COURSE OUTCOME COVERED

CO2: *Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 25

Annexure A

SHORT ANSWER TEST

Duration :60 Mins

On Class

Total Marks:50

Code of Conduct:

I Mathusri P-192311363

(Name/Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Mathusri P

Q.1

Assessment Tool - 2 (CSA1412)

P. Mathusri

192311363

A lexical analyzer produces the token stream:

`id + id * id.`

Explain how the parser validates the syntactic structure

of the expression.

Answer:

The lexical analyzer reads the input string and converts it into a sequence of tokens. For the given input `id + id * id`, the token

stream generated is:

`id, + id`

The

`*`

`id`

`;`

`"`

parser takes this token stream and checks whether it follows the grammar rules. It verifies the correct order of operators and operands, ensuring that each operator has valid operands on both sides (except unary operators).

For this expression, the parser confirms that multiplication has higher precedence than addition. So it first parses

* `id` and then applies `+` with the remaining `id`. Since the expression follows the grammar rules of `id`

arithmetic expressions, the parser validates it as syntactically correct.

Q2.

Given the grammar:

$S \rightarrow a \ Sb \mid \epsilon$

Determine whether the string `aaabb` belongs to the language using derivation.

Answer:

We have the grammar:

$S \rightarrow aSb \mid \epsilon$

We need to generate the string
aaabb.

Leftmost derivation:

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow$
 $aaaSbbb \Rightarrow aaabbb$

Hence, the string aaabb belongs
to the language generated by the
grammar.

Q.3.

Answer:

Remove left recursion from the
grammar: $E \rightarrow E+T \mid T$

The given grammar has
direct left recursion in E

because

E appears on the left side of its
own

E

production.

To remove left recursion,
we can use the standard

method.

Given
grammar:

$EE+TIT$

Here,

$E \rightarrow E + T$

$E \rightarrow T$

We introduce a new

(left recursive part)

(non-left recursive part)

non-terminal E' . The idea is to

factor out the left recursive part.

The new
grammar

$E \rightarrow TE'$

will be:

$$E' \rightarrow +TE' \mid \varepsilon$$

Explanation:

→

$E \rightarrow T E'$ means E first derives T (the part that does not start with E), and then E' handles the repetition of $+T$.

$E' \rightarrow +TE'$ allows one or more occurrences of $+T$.

- $E \rightarrow$
- $E \varepsilon$ allows zero occurrences, which corresponds to the original

production $E \rightarrow T$.

Thus, the left recursion is removed.

Q.4

Answer:

Q.5

Find $\text{FIRST}(A)$ for the grammar:

$A \rightarrow aB \mid \epsilon$

$B \rightarrow b$

To find $\text{FIRST}(A)$, we look at

the productions of A.

$AaB \rightarrow \epsilon$

Q

For AaB : The first symbol is 'a', which is a terminal.

So, $a \in \text{FIRST}(A)$.

For $A \rightarrow \epsilon$: Since A can derive & Empty string),

$\epsilon \in \text{FIRST}(A)$.

Hence, $\text{FIRST}(A) = \{a, \epsilon\}$

Answer:

Determine whether the grammar is suitable for top-down parsing:

$$A \rightarrow Aa \mid b$$

A grammar is suitable for top-down parsing if it is left recursion free.

Given grammar:

$$A \rightarrow Aalb$$

In the production $A \rightarrow Aa$, the non-terminal A appears first symbol on the right-hand side. This is direct left recursion.

as the

Top-down parsers (like
(like recursive descent) cannot handle **left**
recursion because they enter into
an infinite loop while trying to
expand A.

Therefore, this grammar is not
suitable for top-down parsing.

Example of infinite loop:

A Aa Aaa Aaaa . . . (never reaches b)

Hence, the grammar is not
suitable for top-down parsing.

Q.6.

Apply left factoring to the

Answer:"

grammar:

$S \rightarrow \text{while } E \text{ do } S \mid \text{while } E \text{ do } S \text{ else } S$

The two productions of S have a common prefix "while E do S ". So, we left factor it.

Given grammar:

$S \rightarrow \text{while } E \text{ do } S \mid \text{while } E \text{ do } S \text{ else } S$

After left factoring:

$S \rightarrow \text{while } E \text{ do } S S'$

$S' \rightarrow \text{else } S \mid \epsilon$

Here, S' handles the optional part "else S ". If else part is absent, ϵ (empty string) is used.

Q.7. Construct a parse tree for the expression: $\text{id} + \text{id} * \text{id}$

using the grammar:

Answer:

EEEE | id

has higher precedence than +, the expression is parsed

Since

parsed as:

id + (id

id)

+

E

E

id

id

id

*

Q.8.

Answer:

This parse tree shows that `id id` is evaluated first, then added to `id`.

Write the recursive descent procedures for the grammar:

`S → A`

`A → bA | c`

We write one procedure for each non-terminal.

1) Procedure for `S`:

```
s() {  
  if (next symbol is 'a') {
```

2) Procedure for A:

```
A() {  
  if (next symbol is 'b') {  
    match ( 'b');  
    A() ; // Recursive call  
  
  else if (next symbol is 'c')  
    match('c');  
  
  error ( "Invalid input in A");  
  match('a');  
  
  A() ;  
}  
} else  
error("Invalid input in S");  
}
```

else

Q.9.

Given the
grammar:

SAB

$A \rightarrow a \mid \varepsilon$

$B \rightarrow b$

Construct the predictive parsing table.

Answer: First, we compute FIRST sets:

$$\text{FIRST}(A) = \{a, \varepsilon\}$$

$$\text{FIRST}(B) = \{b\}$$

(A can produce 'a' or empty) (B produces 'b')

$$\text{FIRST}(S) = \text{FIRST}(AB) = \{a, b\}$$

Now compute FOLLOW sets:

$$\text{FOLLOW}(S) = \{\$ \}$$

(Because A can produce 'a' or ε .)

If A & , then we take $\text{FIRST}(B) = \{b\}$)

(S is the start symbol)

(From SAB)

(B is at the end)

$$\text{FOLLOW}(A) = \text{FIRST}(B) - \{\varepsilon\} = \{b\} \text{ FOLLOW}(B)$$

$$= \text{FOLLOW}(S) = \{\$ \}$$

Now, we fill the predictive parsing table using the rules:

- If $A \rightarrow \alpha$ and $a \in \text{FIRST}(\alpha)$, put Aa in $M[A, a]$

If $A \rightarrow \epsilon$ and $\epsilon \in \text{FIRST}(\alpha)$, put Aa in $M[A, b]$ for all $b \in \text{FOLLOW}(A)$

M

Terminals

Non-terminals

a

b

$\$$

S

$S \rightarrow AB$

$S \rightarrow AB$

A

Aa

$A \rightarrow \epsilon$

B

$B \rightarrow b$

Q.10.

Hence, the predictive parsing table is constructed successfully.

Demonstrate the first three actions of a

shift-reduce parser for:

`id + id`

Answer: We use the grammar $E \rightarrow E + E \mid id$. The parser begins with empty stack and input string followed by $\$$.

Step

1

Stack

\$

Input

Action

`id + id $`

Shift id

2

\$ id

+ id \$

Reduce $E \rightarrow id$

3

\$ E

+ id \$

Shift +

These are the first three actions of the shift-reduce parser for `id + id`.

Q.11 Identify the handle in the expression:

`id + id`

for the grammar:

`EEE | id`

Answer: A handle is a substring which when reduced to its left-hand side non-terminal

results in the previous right sentential form.

Expression: `id id`

#

`0 id id`

*

(Rightmost derivation in reverse)

→ The rightmost `id` is the handle (matches `id`).

Reduce it to `E`.

So, `id id` $\Rightarrow$ `id * E`

@`id E` → Now the leftmost `id` is the handle. Reduce it to `E`.

↑ ↑

`3 EE` →

So, `id E` $\Rightarrow$ `E * E`

Now the entire string `E E` is the handle

(matches EE) .

Reduce it to E .

Hence, the handles are identified correctly.

Q.12..

Answer

Construct the operator precedence relations for the operators:

$+$, $*$

where ' $*$ ' has higher precedence than ' $+$ '.

In operator precedence parsing, we determine relations between operators using the precedence of operators.

Given :

has higher precedence than ' $+$ '

} .

Let the operators be $\{ +, * \}$

Rules:

An operator of lower precedence yields precedence ($<$) to an operator of higher

precedence.

An operator yields precedence ($>$) to an operator of lower precedence.

Operators of same precedence are related by

Since $*$ has higher precedence than

$<, >$

$+, :$

$*$

$-$

$+$

$>$

$+$ $\wedge$ $\wedge$

$<$

Q.13.

$>$

Hence, the operator precedence relations are

constructed.

Explain how operator precedence parsing handles the expression:

`id + id * id`

Answer: Operator precedence parsing uses precedence relations to decide whether to shift (read next symbol) or reduce.

Q.14.

Given expression: `id + id id (` has

higher precedence than +) Operators: +,

Precedence relations:

Parsing steps (using shift/reduce decisions) :

1 Shift id

2 Next symbol is '+' .

(stack: E)

4

Now top is E and

(stack: E +)

Since operands are on stack, reduce id E .

+> * + >

and

* =

and > + and + > +

(stack: id)

next is '+' . Since $E < +$, shift '+' .

'' .

Next is id. Shift id. (stack: E + id)

5 '*'

Next is. Since has higher precedence than '+' ($+ < *$) ,

shift '*' (stack: E + id *)

6 Next is id. Shift id. (stack: E + id

id)

7 End of input. Now reduce $id \rightarrow E$ (rightmost id) .

(stack: $E + id E$) 8 Since $*$ has higher precedence,

reduce $id E \rightarrow E$. (stack: $E + E$) 9 Finally reduce

$E + E \rightarrow E$. (stack: E)

The final result is E , so the expression is accepted.

What are LR parsers? Illustrate with a simple parsing scenario.

Answer: LR parsers are a class of bottom-up parsers that scan the input from Left to right and produce a rightmost derivation in reverse .

a stack and an LR parsing table built from items. Example

Grammar: $E \rightarrow E + E \mid EE \mid id$

They use

Input string: $id +$

$id + id$ Simple LR Parsing Scenario:

Reduce $E \rightarrow id$, goto state 3

Step

1

Stack

0

Input id + id \$

Action

Shift id (goto state 5)

23456

```

0 id 5
+ id $
OE3
+ id $
OE 3+6
id $
$
OE3+6E9
$
OE 3+ 6 id 5

```

Thus, LR parsers efficiently handle a produce rightmost derivation in reverse

Shift + (goto state 6) Shift id (goto state 5) Reduce $E \rightarrow$ id, goto state 9 Reduce $E \rightarrow E + E$, goto 1 (Accept) wide class of grammars and using shift-reduce actions.

Q.15.

Answer:

Explain the concept of FIRST and FOLLOW sets

with example. **FIRST** set of a symbol a is the set of terminals that can appear as the first symbol in some string derived from a . If $a \Rightarrow^* \epsilon$, then ϵ is also included in **FIRST** (a).

FOLLOW set of a non-terminal A is the set of terminals that can appear immediately to the right of A in some sentential form. If A is the start symbol, then $\$$ is included in **FOLLOW** (A). **Example Grammar:**

FIRST sets:

$\check{E} \rightarrow E + T \mid T$

$T \rightarrow F \mid F$

$T.$

$F \rightarrow (E) \mid id$

FIRST (id) = { id }

FIRST (F) = { $($, id }

FIRST ((E)) = { $)$ }

FIRST (T) = **FIRST** (TF) $\cup$ **FIRST** (F) = **FIRST** (E)
 = **FIRST** ($E+T$) $\cup$ **FIRST** (T) =

$\{+, (, id\}$

FOLLOW sets:

$\{, C, id\}$

$\{+, (, id\} \cup \{+, C, id\}$

Rules used:

(E is start symbol) If A

$FOLLOW(E) = \{ \$,) \}$ $FOLLOW(T) = \{ +, \$,) \}$

$FOLLOW(F) = \{ , +, \$,) \}$

a BB, then $FIRST(B) - \{E\}$

$FOLLOW(B)$

C If A a B or A a BB and $E \in FIRST(B)$,

then $FOLLOW(A) \cup FOLLOW(B)$

3 Add \$ to FOLLOW (start symbol).

Thus, FIRST and FOLLOW sets help in constructing LL (1) parsing table.

**Q.16. What is ambiguity in a grammar?
Give example.**

Answer:

one

A grammar is said to be ambiguous if there

exists at least string in the language that has more than one distinct parse tree (or more than leftmost or rightmost derivation).

than one
one

←

Example Grammar: $E \rightarrow E + E \mid E \mid id$

$id + id \ id$

Consider string: Parse Tree -1:

$(id + id) \ id$

E

Parse Tree -2: $id + (id \ id)$

E I

E

+

id

E-3

id

E-3

E

I

id

E

E

id

id

Q.17.

Answer:)

id

Since the string

the

1

id + id * id

has

grammar is ambiguous.

two different parse trees,

Differentiate between top-down and bottom-up parsing.

Aspect

Direction

Constructs

Technique

Parsers

Top-down Parsing

Starts from start symbol and moves towards input.

Leftmost derivation.

Uses prediction.

Recursive Descent, LL (1) Parser

Backtracking May require backtracking. Error

Detection Detects errors early.

Bottom-up Parsing Starts from input and builds towards start symbol.

Rightmost derivation in reverse.

Uses reduction .

Shift-Reduce Parser, LR Parser **No** backtracking.

Detects errors late .

Both methods are useful; top-down is easier to implement, while bottom-up is more powerful.

CQ.18.

Answer:

Explain the role of symbol table in a compiler.

The symbol table is a data structure used

by the compiler to store information about the identifiers (names) used in the program.

It is created during lexical analysis and used in all subsequent phases. Each entry in the symbol table contains attributes of

Roles :

an

1 Stores names of variables, functions, arrays, constants, etc.

identifier.

2 Keeps information about data type, scope, value, and memory location. (3 Helps in type checking and semantic analysis.

4 Detects errors such as redeclaration, undeclared identifier, type mismatch. C Used for code generation (gives address of variables, etc.).

6 Maintains scope and lifetime information of identifiers. Example Symbol Table:

Name

Type

a

Variable

Scope Global

Data Type Address/Value

Other Info

int

1000

Initialized

b

Variable

Local

float

2000

sum()

Function

Global

void

3000

MAX

Constant

Global

int

10

2 parameters Read-only

Q.19.

Answer:

Q.20.

Answer :

Hence, the symbol table plays a vital role in supporting the various phases of compilation and ensures efficient processing of identifiers.

What is error handling in a compiler? List different types of errors. Error handling in a compiler refers to the detection, reporting, and recovery from errors that occur in the source

program during compilation. A good compiler should detect as many errors as possible and report meaningful messages to help the programmer.

Types of Errors:

Lexical Errors

7 Syntax Errors

: Errors in the structure of tokens.

Example: Invalid character, incorrect number format. : Errors in the grammatical structure of the program. Example:

Missing semicolon, unmatched parentheses. (3) Semantic Errors: Errors in the meaning of statements.

4

Example: Type mismatch, undeclared variable. Run-time

Errors: Errors that occur during execution.

Example: Division by zero, array out of bounds.

3 Logical Errors : Errors in the logic of the program (not detected by compiler).

Example: Wrong formula, incorrect condition.

The compiler should report the error with line number and description, and also try to continue compilation to find

more errors.

Write short notes on Intermediate Code Generation.

Intermediate Code Generation is the phase in a compiler where the source program is translated into an intermediate representation (IR) that is easier to analyze, optimize, and generate target code.

Characteristics of IR:

Machine independent.

Easy to generate and translate.

- Suitable for optimization.

Forms of Intermediate Code:

Three Address Code (TAC)

2 Quadruples

Triples

3 Syntax Trees

4 Indirect Triples

Postfix Notation

Example:

Expression: $a = b + c + d$ Three Address Code (TAC) :

$t1 = c + d$

$t2 = b + t1$

$a = t2$

Intermediate code acts as a bridge between the front-end and back-end of the compiler.

Q.21.

Answer:

Explain Code Optimization and list its goals and types.

Code Optimization is the process of improving the intermediate or target code generated by the compiler so that the program runs faster, uses less memory and consumes fewer resources without changing the meaning of the program. Goals of Code Optimization:

1 Reduce the execution time.

2 Reduce the memory usage.

or behavior of the program.

3 Improve the throughput of the program.

Reduce the power consumption. 5 Do not change the

meaning Types of Code Optimization: 1) Machine Independent (IR Level)

Loop optimization (loop invariant code motion, loop unrolling, loop fusion).

- Common subexpression elimination.
- Constant folding and propagation.
- Dead code elimination.

Strength reduction.

2) Machine Dependent (Target Code Level)

Register allocation and assignment.

- Instruction scheduling.

Peephole optimization.

- Use of addressing modes.

Exploiting hardware features.

Thus, code

optimization improves the efficiency of the program while preserving its behavior.

Q.22 . . What is backpatching? Explain with example.

Answer :

Let,

Backpatching is a technique used in code generation to handle jumps (like goto, if, while) whose target addresses are not known at the time of code generation. A list of such jump locations is maintained and later "patched" with the correct target address when it becomes known. Example:

Ex 9

if E1 then S1 else S2

5555

B beginning address of E1 S1 → beginning address of S1
S2 → beginning address of S2

→ address after S2

S

The code generated with backpatching is :

Instruction

Address

B

code for E1

B1

if false goto -

B2

code for S1

B3

goto

S2

code for S2

S

Next instruction

Comment

Evaluate condition E1

Jump to S2 if E1 is false (to be backpatched) Then part

Jump to S (to be backpatched)

Else part (target of B1)

Statement after if-else (target of B3)

Thus, backpatching allows the compiler to generate correct jumps target addresses are not known initially.

Q.23. Explain common subexpression elimination with example.

Answer:

Backpatching steps:

1 Initially, B1 and B3 have

blanks.

When S2 address is known, backpatch B1 with S2.

3 When S address is known,

backpatch B3 with s. even when

Common Subexpression Elimination (CSE) is an optimization technique used to eliminate repeated calculations of the same expression. If a subexpression has already been computed and its operands have not changed, its previous value is reused instead of recomputing it.

Example:

Without CSE

```
t1 = a + b
t2 = c + d
t3 = a + b
t4 = t1
t5 = t3 + t2
```



With CSE

```
t1 = a + b
t2 = c + d
t4 = t1 = t2
t5 = t1 + t2 (reuse t1, no recomputation)
```

Benefits:

- Reduces number of computations.
- Saves execution time.
- Saves memory and registers.

CSE improves efficiency by avoiding redundant evaluations of the same expressions.

Q.24.

Answer:

What is a

syntax directed definition? Explain

SDD with example.

A **Syntax Directed Definition (SDD)** is a formal way of associating attributes and semantic rules with the grammar productions. It is used to specify the meaning of the source declarative manner.

SDD consists of:

Attributes

- Semantic Rules

Example:

program

Values associated with
grammar symbols.

in a

Rules that compute the values of attributes.

Consider the grammar for arithmetic
expression:

$E \rightarrow E_1 + T$

$E_1 \rightarrow T$

1

T

$T \rightarrow T_1 * F$

$T_1 \rightarrow F$

1

F

$F \rightarrow (E)$

1

id

Let val be the synthesized attribute representing the value of an expression.

Semantic rules :

$E1.val + T.val$

$E \rightarrow E1 + T$

{

$E.val$

=

$E \rightarrow E1 - T$

{

$E.val$

$E1.val$

$T.val$

$E \rightarrow T$

{

$E.val$

=

$T \rightarrow T1 * F$

{

$T.val$

=

$T1.val$

$T \rightarrow$

$T1/F$

{

T.val
T.val }

T1.val / F.val }
F.val }

$T \rightarrow F$
{
T.val
T.val
=
F.val }

$F \rightarrow (E)$
{
F.val =
E.val }

$F \rightarrow id$
{ F.val =
id.lexval }

(lexval is the value of identifier)

Thus, SDD helps to evaluate analysis during parsing.

expressions and perform other semantic

Q.25 .

Answer :

an

error, reporting it, and can be found in a

Discuss error recovery techniques in top-down parsing. Error recovery is the process of detecting continuing the parsing so that more single run. In top-down parsing (LL parsing), common error recovery techniques are:

1 . Panic Mode Recovery:

.

.

.

errors

Discard input symbols until a symbol from a synchronizing set is found. Synchronizing set usually contains tokens like ';', '}', ')', etc .

This allows the parser to resume parsing from a known good state. Simple but may skip a

may skip a large portion of correct input.

2. Phrase Level Recovery:

Make small modifications to the input so that parsing so that parsing can continue.

Possible modifications: insert a missing symbol, delete an extra symbol, or replace a symbol.

Example: If ') ' is missing, insert it and continue.

- More accurate but more complex to implement.

3. Error Productions:

- Add special productions in the grammar to handle common

errors.

These productions produce the `error` token and allow parsing to continue. Example: add $A \rightarrow \text{error}$ to handle unexpected symbols.

Helps in giving better and more meaningful better and more meaningful `error` messages.

Thus, error recovery techniques help the parser to handle errors gracefully and continue parsing to report multiple errors in one pass.

RUBRICS FOR CALCULATION

Criteria

Weightage

25% Understanding of Concepts

Level 4- **Exemplary**

Demonstrates thorough,

accurate understanding

of concepts

Detailed,

well-explained

Level 3- **Proficient**

Good understanding **with minor** gaps

Clear **explanation with** adequate details

Level 2- **Developing**

Basic understanding **with some misconceptions**

Limited **explanation**; lacks depth

Level 1- **Beginning**

Poor understandi

ng; major misconcepti

ons

Poor or

unclear explanation

Explanation

25%

answers with

relevant

examples

Effectively

applies

Application

20%

concepts with

strong

Applies concepts with minor errors

Organization

15%

Accuracy

10%

Clarity

5%

analytical

ability Well-structured **answers** with logical **flow and coherence**

Completely accurate and

covers all required points

Clear, neat,
professional
presentation

Organized **with** minor **issues** in flow

Mostly accurate with minor **omissions**

Understandable
with **minor**
issues

Limited application with **weak analysis**

Basic **structure, lacks** clarity

Partially correct **with missing** points

Limited clarity,

readability **issues**

No application or incorrect **analysis**

Poor organization , incoherent

answers Incorrect

answers with major

gaps Poorly written,

difficult to

understand